

Sprawozdanie z Laboratorium 6

Podstawy konfiguracji środowiska chmurowego i automatyzacji

Mateusz Hypta

Politechnika Wrocławska

Przedmiot: Praktyki Programowania

Prowadzący: mgr inż. Michał Jaroszczuk

24 maja 2026

Spis treści

1	Cel laboratorium	2
2	Wstęp teoretyczny	2
2.1	Kluczowe aspekty DevOps i konteneryzacji	2
3	Opis wykonanych zadań	3
3.1	Aplikacja testowa (Kalkulator)	3
3.2	Konteneryzacja środowiska (Docker)	3
3.3	Automatyzacja procesów – Lokalny Potok CI/CD	3
4	Wnioski	4

1 Cel laboratorium

Celem zajęć było poznanie chmury, podstaw metodyki DevOps oraz automatyzacji testów i wdrożeń, czyli potoków CI/CD. Niestety, przez problemy techniczne z logowaniem i autoryzacją na studenckim koncie Microsoft Azure, nie dało się zrobić zadań w chmurze. Z tego powodu całą część praktyczną zrobiłem lokalnie na komputerze przy użyciu Dockera. Dzięki temu udało się odtworzyć dokładnie te same mechanizmy automatyzacji i izolacji środowiska, o które chodziło w oryginalnym zadaniu.

2 Wstęp teoretyczny

2.1 Kluczowe aspekty DevOps i konteneryzacji

DevOps polega na połączeniu pracy programistów (Dev) i administratorów odpowiedzialnych za utrzymanie systemów (Ops). Głównym celem jest tutaj automatyzacja wszystkiego, co się da – od sprawdzania kodu po jego wydawanie (potoki CI/CD).

W takim podejściu bardzo ważny jest Docker i konteneryzacja. Pozwala ona zamknąć aplikację wraz ze wszystkimi potrzebnymi bibliotekami w małym, odizolowanym kontenerze. W przeciwieństwie do zwykłych maszyn wirtualnych, kontenery nie potrzebują całego systemu operacyjnego dla siebie, bo korzystają z systemu komputera-gospodarza. Dzięki temu działają super szybko i zużywają mało zasobów. Najważniejsza zaleta jest taka, że aplikacja w kontenerze działa wszędzie tak samo – na komputerze programisty, serwerze testowym czy w chmurze, co rozwiązuje odwieczny problem pod tytułem „u mnie działa, a na serwerze nie”.

3 Opis wykonanych zadań

3.1 Aplikacja testowa (Kalkulator)

Do testów automatyzacji posłużył prosty kalkulator napisany w Pythonie (`calc.py`) oraz plik z testami jednostkowymi (`test_calc.py`), które uruchamia się za pomocą narzędzia `pytest`.

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b
```

Listing 1: Kod źródłowy kalkulatora (`calc.py`)

3.2 Konteneryzacja środowiska (Docker)

Żeby aplikacja nie zależała od tego, co mam zainstalowane na systemie, stworzyłem plik `Dockerfile`. Przygotowuje on czyste środowisko z Pythonem i instaluje potrzebne rzeczy.

```
FROM python:3.9-alpine  
WORKDIR /app  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
COPY . .  
CMD ["python", "calc.py"]
```

Listing 2: Plik konfiguracyjny `Dockerfile`

Budowanie obrazu i samo odpalenie testów w kontenerze wykonałem poniższymi komendami w terminalu:

```
docker build -t local-calc-app .  
docker run --rm local-calc-app pytest
```

3.3 Automatyzacja procesów – Lokalny Potok CI/CD

Żeby zasymulować działanie automatycznych potoków z chmury (takich jak Azure Pipelines), napisałem skrypt w Bashu. Robi on automatycznie to, co normalnie działałoby się na serwerze CI/CD:

1. Przygotowuje pliki i buduje kontener w Dockerze.

2. Uruchamia automatyczne testy (pytest) wewnątrz kontenera.
3. **Zadanie dodatkowe (Kopia zapasowa):** Jeśli testy przejdą pomyślnie, skrypt sam tworzy folder backup i kopiuje tam plik z kalkulatorem.

```
#!/bin/bash
set -e

echo "===_KROK_1:_Budowanie_środowiska_kontenerowego_==="
docker build -t devops-pipeline-image .

echo "===_KROK_2:_Uruchamianie_testów_jednostkowych_==="
docker run --rm devops-pipeline-image pytest

echo "===_KROK_3:_Wykonywanie_zadania_dodatkowego_(Backup)_==="
mkdir -p backup
cp calc.py backup/
echo "Backup_wykonany_pomyślnie._Potok_zakończył_się_sukcesem!"
```

Listing 3: Skrypt automatyzujący potok CI/CD i backup

4 Wnioski

Używanie Dockera i podejścia DevOps pozwala uniezależnić aplikację od komputera, na którym jest uruchamiana, dzięki czemu proces budowania i testowania jest zawsze taki sam. Mimo że przez błąd uwierzytelniania nie udało się wejść do chmury Azure, Docker pozwolił mi odtworzyć identyczne reguły i kroki automatyzacji na dysku lokalnym. Pokazuje to, że nieważne czy pracujemy w chmurze, czy na własnym komputerze – same zasady automatyzacji i pisania skryptów w DevOps wyglądają dokładnie tak samo.